

# COURS PROGRAMMATION MOBILE

---

Développer des applications modernes avec Flutter & Dart



Dr MORIE Wielfrid  
[moriemaho@gmail.com](mailto:moriemaho@gmail.com)  
07 09 36 98 09



# Objectifs

---



- Maîtriser les fondamentaux du SDK Flutter
- Développer avec le langage Dart
- Développer des applications Android & iOS performantes
- Effectuer la persistance des données avec sqflite
- Connaître les bonnes pratiques de développement



# Références

---



- **Liens**

- <https://docs.flutter.dev/>
- <https://dartlangfr.github.io/>

- **Livres**

1. Flutter Développez vos applications mobiles multiplateformes avec Dart (Julien TRILLARD)
2. Flutter Complete Reference (Alberto Miola)

- **Chaines YouTube**

- FlutterWay
- Academind





# Historique



**Motorola V3**  
**2005**  
**App interne**  
**Jeux en Java**



**Iphone 1**  
**2007**  
**iOS**  
**Apple store**



**Samsung Galaxy S1**  
**2010**  
**Android 2.1**  
**Playstore**





# Historique...

---



**Android**  
Google



**Windows phone**  
Microsoft



**iOS**  
Apple



**BBOS**  
Blackberry





# Outils & langages

---



- Kotlin, Java
- Android Studio (SDK Android)
- Windows, Linux, MacOS



- Swift
- Xcode (SDK iOS)
- MacOS

Développement Native





# Framework

## WebView



Javascript

Composants

WebView

Plateforme



## Close to native



React Native

Javascript

Composants

Pont

Java



Swift



## Widget rendering



Dart

Composants

Widget crée

Widget  
Rendu



Widget  
Rendu







# Framework...

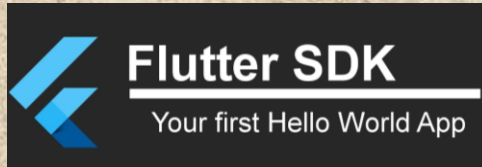


React Native 0.68

Flutter 3

|                           |                                     |                                       |
|---------------------------|-------------------------------------|---------------------------------------|
| Initial release           | May 2017                            | March 2015                            |
| Backed by                 | Google                              | Facebook                              |
| Programming language      | Dart                                | Java Script                           |
| Platform support (stable) | Android, iOS                        | Android, iOS, Web Apps                |
| App performance           | Close to native                     | Fairly robust                         |
| Open Source               | Yes                                 | Yes                                   |
| Documentation             | Extensive                           | Extensive                             |
| UI                        | Proprietary customized widgets      | Native components                     |
| Community & support       | Limited, fast growing               | Extensive                             |
| 60+ fps support           | Yes                                 | Requires workarounds                  |
| Code reusability          | Up to 90%                           | Up to 90%                             |
| JIT, AOT compilation      | Yes                                 | No                                    |
| Used by                   | Google, Alibaba, Tencent, Reflectly | Facebook, Instagram, Uber, Salesforce |





Ajouter le dossier bin dans les variables d'environnement  
Entrer la commande « **Flutter Doctor** »



Android Studio

Téléchargement des composants  
Téléchargement du SDK CLI tools



Visual Studio Code

Dart  
Flutter  
Bracket pair colored  
Awesome flutter code





# Syntaxe de base

---

- Fonction principale « main » obligatoire
- Les délimiteurs de fonctions sont les accolades « { } »
- Chaque instruction se termine par un point virgule « ; »
- Les commentaires sont donnés par :
  - // : commentaires une ligne
  - /\* \*/ : commentaires sur plusieurs lignes
- Indentation avec 2 espaces





# Convention de codage

| Identifiant                                                               | Style                     |
|---------------------------------------------------------------------------|---------------------------|
| Variable :<br>Constantes :<br>Fonctions :<br>Propriétés :<br>Paramètres : | lowerCamelCase            |
| Classes :                                                                 | UpperCamelCase            |
| Bibliothèques et Fichiers .dart                                           | lowercase_with_underscore |

```
class HttpConnection (String plexGame ){}  
class DBIOPort {}  
class TVVcr {}  
class MrRogers {}  
  
var httpRequest = ...  
var uiHandler = ...  
var userId = ...  
Id id;
```





# Mots clés

|                               |                                |                               |                             |
|-------------------------------|--------------------------------|-------------------------------|-----------------------------|
| <u>abstract</u> <sup>2</sup>  | <u>else</u>                    | <u>import</u> <sup>2</sup>    | <u>show</u> <sup>1</sup>    |
| <u>as</u> <sup>2</sup>        | <u>enum</u>                    | <u>in</u>                     | <u>static</u> <sup>2</sup>  |
| <u>assert</u>                 | <u>export</u> <sup>2</sup>     | <u>interface</u> <sup>2</sup> | <u>super</u>                |
| <u>async</u> <sup>1</sup>     | <u>extends</u>                 | <u>is</u>                     | <u>switch</u>               |
| <u>await</u> <sup>3</sup>     | <u>extension</u> <sup>2</sup>  | <u>late</u> <sup>2</sup>      | <u>sync</u> <sup>1</sup>    |
| <u>break</u>                  | <u>external</u> <sup>2</sup>   | <u>library</u> <sup>2</sup>   | <u>this</u>                 |
| <u>case</u>                   | <u>factory</u> <sup>2</sup>    | <u>mixin</u> <sup>2</sup>     | <u>throw</u>                |
| <u>catch</u>                  | <u>false</u>                   | <u>new</u>                    | <u>true</u>                 |
| <u>class</u>                  | <u>final</u>                   | <u>null</u>                   | <u>try</u>                  |
| <u>const</u>                  | <u>finally</u>                 | <u>on</u> <sup>1</sup>        | <u>typedef</u> <sup>2</sup> |
| <u>continue</u>               | <u>for</u>                     | <u>operator</u> <sup>2</sup>  | <u>var</u>                  |
| <u>covariant</u> <sup>2</sup> | <u>Function</u> <sup>2</sup>   | <u>part</u> <sup>2</sup>      | <u>void</u>                 |
| <u>default</u>                | <u>get</u> <sup>2</sup>        | <u>required</u> <sup>2</sup>  | <u>while</u>                |
| <u>deferred</u> <sup>2</sup>  | <u>hide</u> <sup>1</sup>       | <u>rethrow</u>                | <u>with</u>                 |
| <u>do</u>                     | <u>if</u>                      | <u>return</u>                 | <u>yield</u> <sup>3</sup>   |
| <u>dynamic</u> <sup>2</sup>   | <u>implements</u> <sup>2</sup> | <u>set</u> <sup>2</sup>       |                             |





# Hello World

---

```
void main() {  
  print('Hello World!');  
}
```





# Variables

| Type           | Domaine                                                                 | Exemple                                           |
|----------------|-------------------------------------------------------------------------|---------------------------------------------------|
| var            | Typage dynamique (faible)                                               | var keyLabel = 'poor'                             |
| num            | Nombre (réel ou entier)                                                 | num mainFlow = 3.5                                |
| int            | Entier                                                                  | int bigFlow = 3                                   |
| double         | Réel                                                                    | double lowFlow = 8.12                             |
| String         | Chaine de caractère                                                     | String carName = 'Mercedes'                       |
| bool           | Booléen                                                                 | bool isChecked = true                             |
| List           | Liste ou tableau                                                        | List <int> myList = [1,2,3]                       |
| Map            | Dictionnaire                                                            | Map<int, String> nobleGases = { 2: 'helium' }     |
| Set            | List de valeur unique                                                   | Set<String> ipxNames = { 'fluorine', 'chlorine' } |
| <b>const</b>   | <b>Constante</b>                                                        | <b>const price = 12.02</b>                        |
| <b>final</b>   | <b>Dernière valeur</b>                                                  | <b>final price = 14</b>                           |
| <b>dynamic</b> | <b>La valeur attendu peut être dynamic. Exemple avec les Map et Set</b> |                                                   |





# Fonctions

## Fonction sans valeur de retour : Procédure

```
void bonMessage() {  
    print('Bienvenue');  
}
```

```
void main() {  
    bonMessage();  
}
```

## Fonction avec valeur de retour

```
int somme(int a, int b) {  
    return a+b;  
}
```

```
void main() {  
    int x;  
    x = somme(3, 6);  
}
```





# Opérateurs

| Description        | Opérateurs                |
|--------------------|---------------------------|
| Multiplication     | * / % ~/                  |
| Addition           | + -                       |
| Et                 | &                         |
| XOR                | ^                         |
| Ou                 |                           |
| Test de condition  | >= > <= < as is is! == != |
| Et logique         | &&                        |
| Ou logique         |                           |
| Si null            | ??                        |
| Condition ternaire | expr1 ? expr2 : expr3     |
| Affectation        | = *= /= += -= &= ^= etc.  |





# Structures de contrôle : Structure conditionnelle - if

```
if (weather == isRaining) {  
  print('Pluie');  
} else if (weather == isSnowing) {  
  print('Neige');  
} else {  
  print('sunlight');  
}
```





# Structures de contrôle : Structure de répétition - **for**

## Accès par indices

```
for (var i = 0; i < 5; i++) {  
  print(i);  
}
```

## Accès par l'objet itérable

```
var collection = [0, 1, 2];  
for (var x in collection) {  
  print(x);  
}
```

## Méthode **forEach()**

```
var collection = [1, 2, 3];  
collection.forEach(print);
```





# Structures de contrôle : Structure de répétition - While

---

```
while (isDone) {  
    print('It is OK');  
}
```

```
do {  
    printLine();  
} while (!atEndOfPage);
```





# POO : Classes

```
Class MyPoint {
```

```
  num x;
```

```
  num _y; ← Variable Private
```

```
  Point(num x, num y) { ← Constructeur de classe qui  
    peut être écrit autrement
```

```
    this.x = x;
```

```
    this.y = y;
```

```
  }
```

```
}
```

↓

```
Point(this.x, this.y);
```





# POO : Getters & Setters

Les Getters et les Setters peuvent être créés à l'aide des mots clés **Get** et **Set**. Cependant, il faut savoir que chaque variable d'instance possède un getter implicite, ainsi qu'un setter si nécessaire.

```
double get right () {  
    return left + width;  
}
```

Getter de la propriété right

```
set right(double value) {  
    left = value - width;  
}
```

Setter de la propriété right





# POO : Héritage

Utilisez **extends** pour créer une sous-classe, et **super** pour faire référence à la super-classe

```
class Television {  
    void allumer() {  
        _eclairerLecran();  
    }  
}
```

```
class TelevisionIntelligente extends Television {  
    void allumer() {  
        super.allumer();  
        _demarrerInterfaceReseau();  
        _mettreAJourApplications();  
    }  
}
```





# POO : Redefinition

Les sous-classes peuvent remplacer les méthodes d'instance.  
L'annotation `@override`.

```
class Television {  
  set contrast(int value) {...}  
}
```

```
class SmartTelevision extends Television {  
  @override  
  set contrast(num value) {...}  
}
```





# Les collections : Iterable

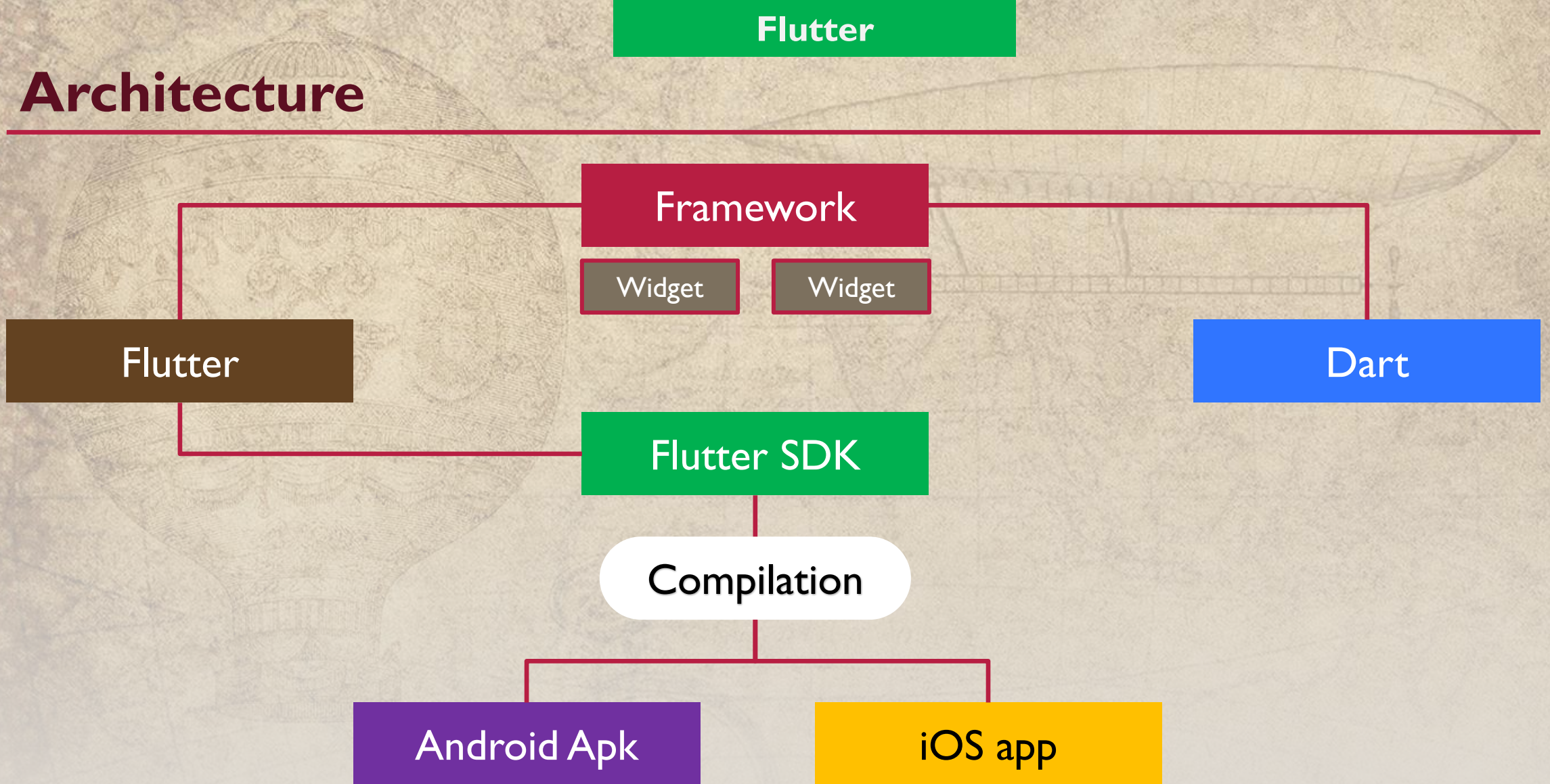
Les collections sont un ensemble d'éléments itérable, i.e., on peut les parcourir.

List, Set, Map.

|                                                                        |                                                                                                           |
|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>var iterable = ['Salad', 'Popcorn', 'Toast']</code>              | Déclaration d'un itérable (List)                                                                          |
| <code>for (final element in iterable)</code>                           | Parcours d'une liste, ou Map ou Set                                                                       |
| <code>iterable.first</code>                                            | Accéder au premier élément de l'itérable                                                                  |
| <code>iterable.last</code>                                             | Accéder au dernier élément de l'itérable                                                                  |
| <code>iterable.firstWhere((element) =&gt; element.length &gt; 5</code> | Récupérer le premier élément selon une condition                                                          |
| <code>iterable.where((element) =&gt; element == 12</code>              | Récupérer tous les éléments selon une condition                                                           |
| <code>numbers.map((number) =&gt; number * 10)</code>                   | La fonction map change tous les éléments de l'itérable par le résultat d'une fonction passée en paramètre |



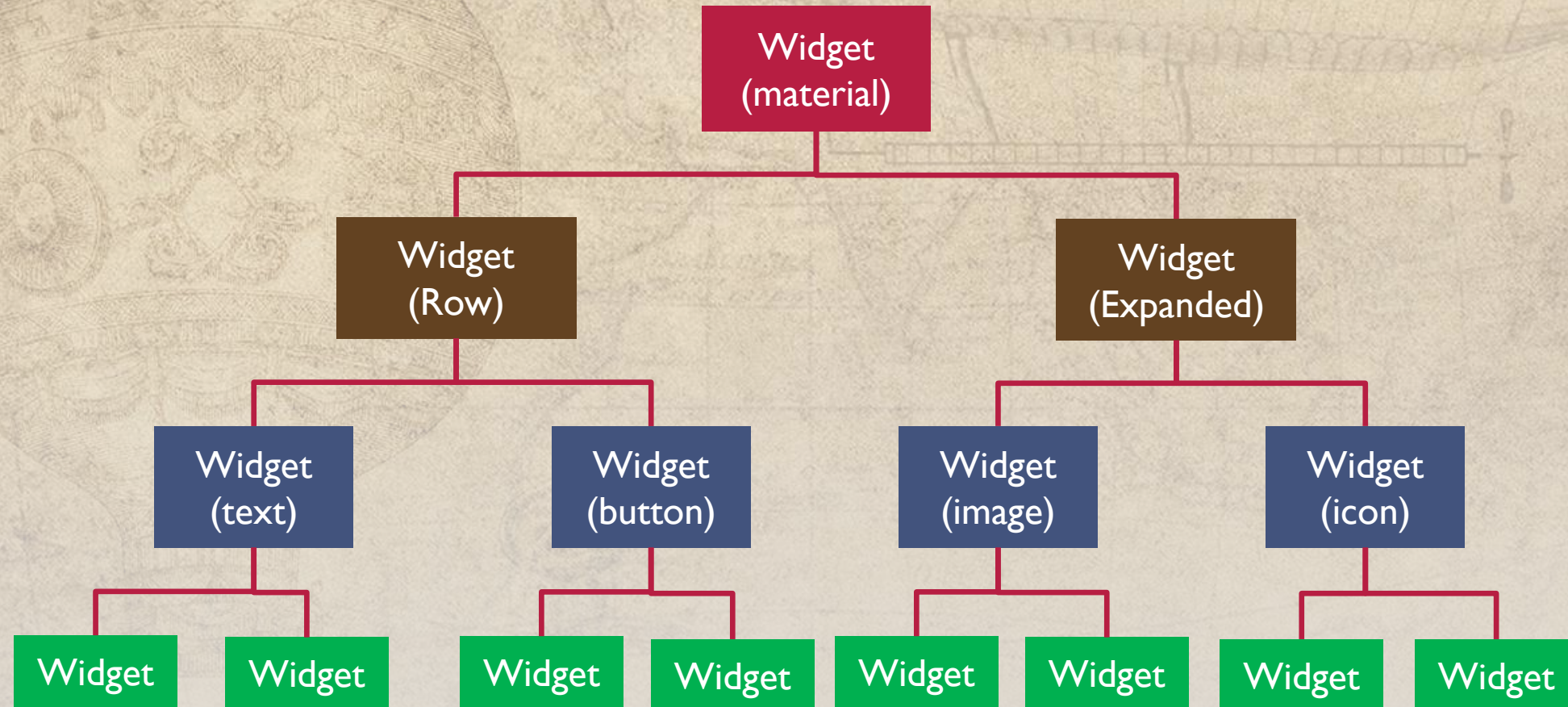
# Architecture







# Paradigme







# Installation

Flutter.dev

Flutter SDK

Android studio

Visual Studio Code

~ 4Gb de téléchargement

Suivez les instructions de cette vidéo toujours d'actualité pour la version

**Flutter 3.7** (mars 2023) :

**COMMENT INSTALLER FLUTTER SUR WINDOWS - Technovore**

<https://www.youtube.com/watch?v=RTeSjLDjgds>





# Installation

---

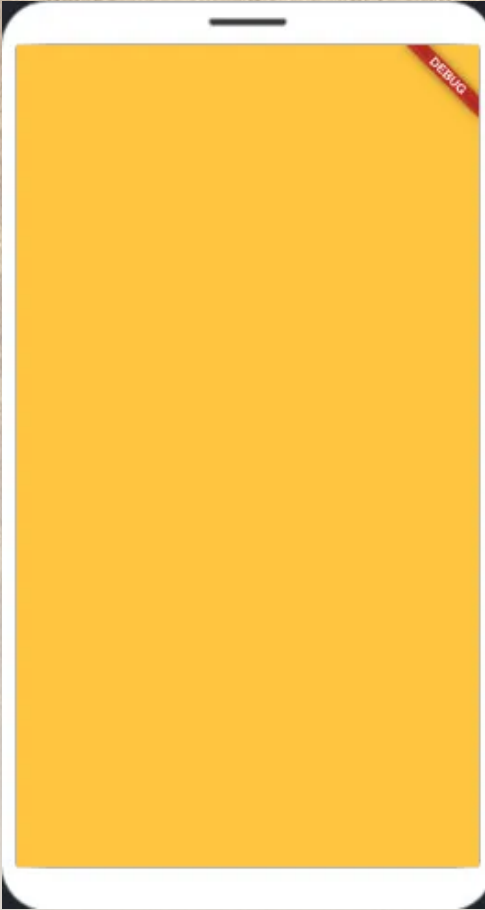
Installons ensemble si  
vous le voulez bien !





# Widgets de page

## MaterialApp



Le principal widget qui englobe la plupart des widgets généralement requis pour les applications.

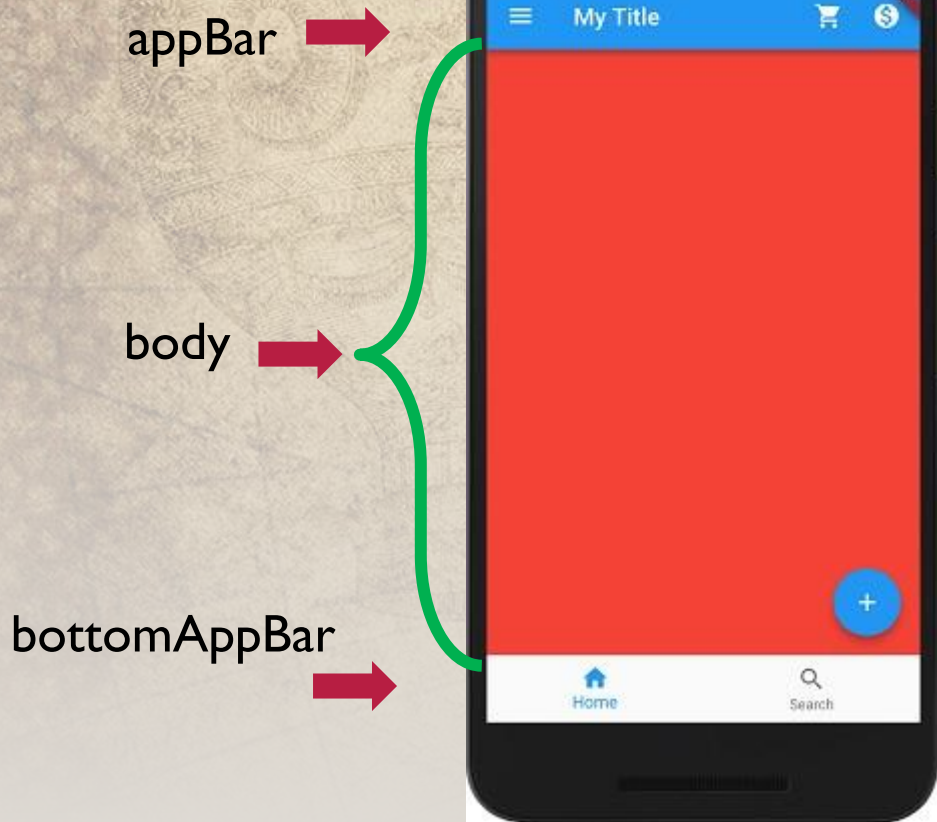
```
debugShowCheckedModeBanner : false  
theme  
  Brightness : Brightness.Dark  
  primaryColor : Colors.yellow  
home : ...
```





# Widgets de page

## Scaffold



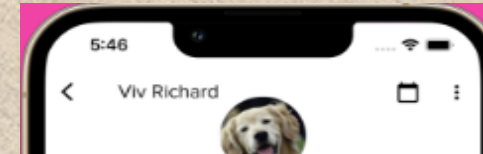
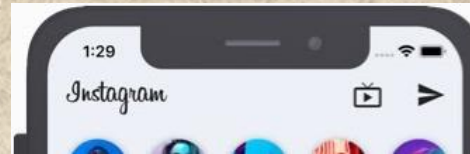
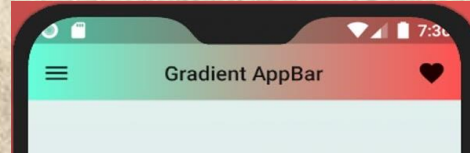
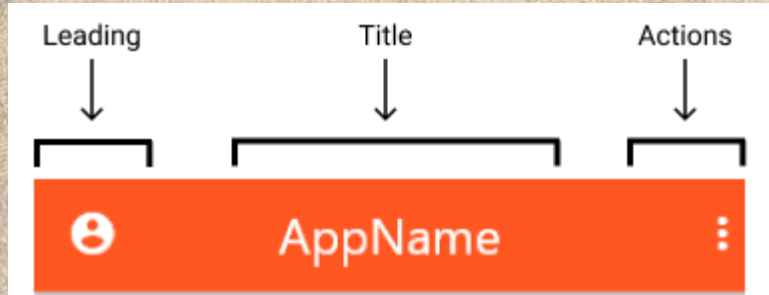
Mettre en œuvre la structure de base de la conception Material design.  
Style recommandé pour les application Android.

```
appBar :AppBar()  
Body :Widget()  
floatingActionButton : FloatingActionButton()  
bottomNavigationBar : [Widget()]
```



# AppBar

**AppBar** est la barre d'application située au dessus du contenu principal.



- **actions** → List<Widget>?
- **leading** → Widget?
- **title** → Widget?

```
12 );  
13 }  
14 }  
15  
16 class AppBarExample extends StatelessWidget  
17   const AppBarExample({super.key});  
18  
19   @override  
20   Widget build(BuildContext context) {  
21     return Scaffold(  
22       appBar: AppBar(  
23         title: const Text('AppBar Demo'),  
24         actions: <Widget>[  
25           IconButton(  
26             icon: const Icon(Icons.add_alert),  
27             tooltip: 'Show SnackBar',  
28             onPressed: () {
```

AppBar Demo

DEBUG

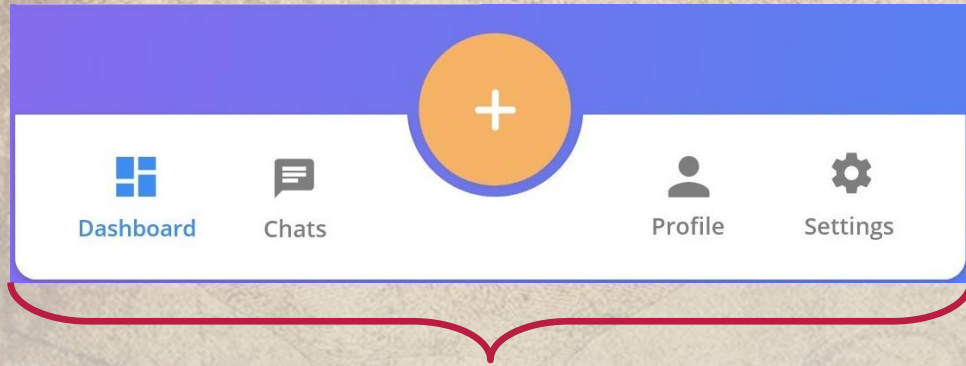
This is the home page





# bottomNavigationBar

**bottomNavigationBar** est la barre d'application située en dessous du contenu principal.



bottomNavigationBarItem → <Widget>?

```
),  
body: Center(  
  child: _widgetOptions.elementAt(_selected  
),  
bottomNavigationBar: BottomNavigationBar(  
  items: const <BottomNavigationBarItem>[  
    BottomNavigationBarItem(  
      icon: Icon(Icons.home),  
      label: 'Home',  
    ),  
    BottomNavigationBarItem(  
      icon: Icon(Icons.business),  
      label: 'Business',  
    ),  
    BottomNavigationBarItem(  
      icon: Icon(Icons.school),  
      label: 'School',  
    ),  
  ],  
  currentIndex: _selectedIndex,  
  selectedItemColor: Colors.amber[800],  
),
```

BottomNavigationBar ...

DEBUG

Index 0: Home

Home

Business

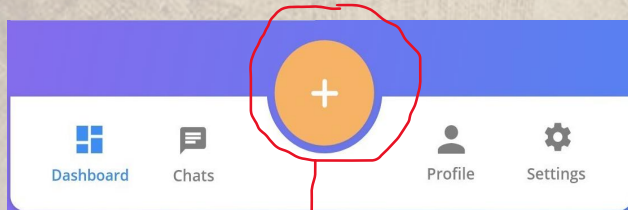
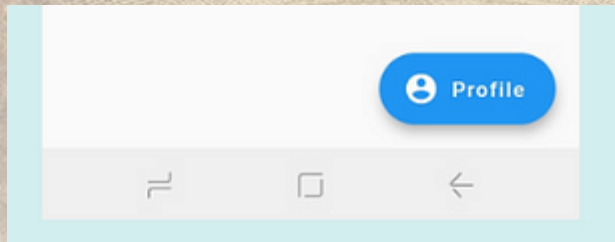
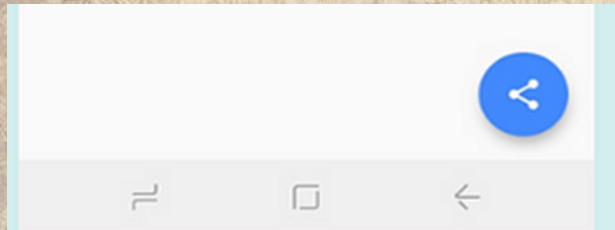
School





# floatingActionButton

**floatingActionButton** est un bouton flottant généralement situé au dessus du bottomNavigationBar. Il est soit à droite, soit au milieu.



FAB

```
is FabExample extends StatelessWidget {  
  const FabExample({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: const Text('FloatingActionButton Sa  
      ),  
      body: const Center(child: Text('Press the  
      floatingActionButton: FloatingActionButton(  
        onPressed: () {  
          // Add your onPressed code here!  
        },  
        backgroundColor: Colors.green,  
        child: const Icon(Icons.navigation),  
      ),  
    );  
  }  
}
```

FloatingActionButton ...

DEBUG

Press the button below!



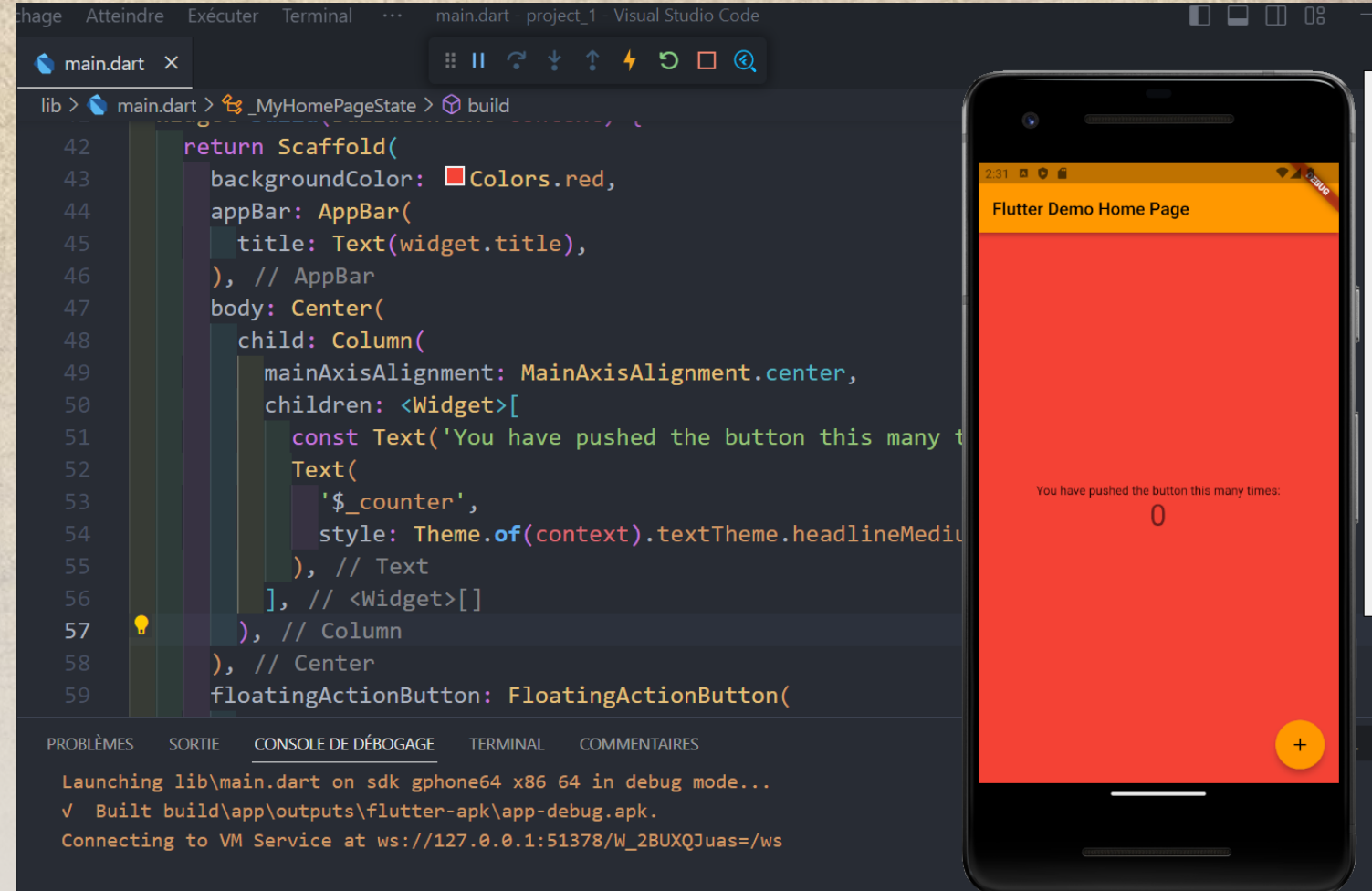
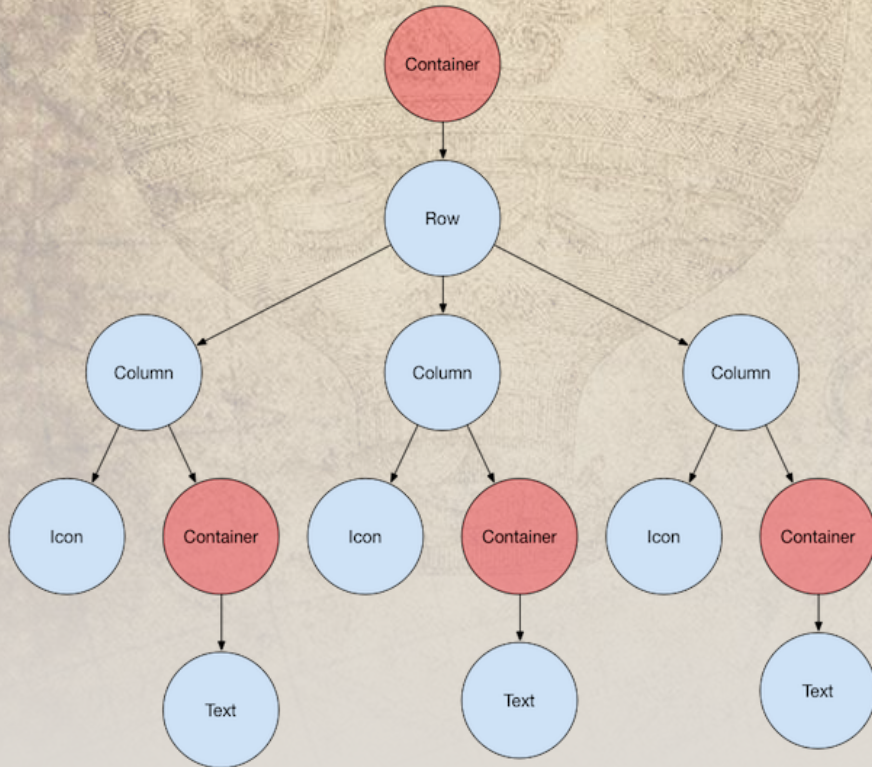




# body

Body est le corps de notre application. Il contiendra tous les widgets pour faire fonctionner notre application.

- Un widget
- Pas de propriétés
- Entre le appBar et le bottomNavigationBar







# Widgets Layout

Les Layout sont des widgets d'organisation du design de l'application. Ils permettent de gérer la disposition des autres widgets sur l'application. De ce fait ils peuvent embarquer plusieurs widgets disposés selon leur principale axe. Ils attendent donc une liste de widgets dans la propriété **children qui est obligatoire.**

**children** : <Widget>[...]

**crossAxisAlignment**: CrossAxisAlignment.start

**mainAxisSize**: MainAxisSize.min

**mainAxisAlignment**: MainAxisAlignment.center

**textBaseline** :TextBaseline

Column



Row



Stack







# Widgets Layout : *Column*

Le widget `Column` permet de disposer les widgets en son sein les uns en dessous des autres. Son axe principale (*mainAxis*) est la verticale et son axe secondaire (*crossAxis*) l'horizontale. Le widget `Column` n'a pas de barre de défilement. Ainsi, s'il y a plus de contenu que la taille de l'écran (le widget parent), une erreur d'affichage se produira. Il faut se pencher dans ce cas vers d'autres widgets adaptés.

Axe principal : verticale  
Axe secondaire : horizontale

Column



## Propriétés :

*mainAxisAlignment* = Alignement sur l'axe vertical

*mainAxisSize* = Hauteur de la colonne

*crossAxisAlignment* = Alignement sur l'axe horizontal

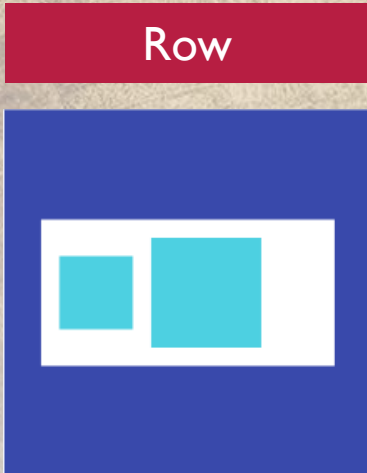




# Widgets Layout : Row

Le widget *Row* permet de disposer les widgets en son sein les uns à côté des autres. Son axe principale (*mainAxis*) est l'horizontale et son axe secondaire (*crossAxis*) la verticale. Le widget *Row* n'a pas de barre de défilement également. Ainsi, s'il y a plus de contenu que la largeur de l'écran (le widget parent), une erreur d'affichage se produira tout comme le widget *Column*. Il faut se pencher dans ce cas vers d'autres widgets adaptés.

Axe principal : horizontale  
Axe secondaire : verticale



## Propriétés :

*mainAxisAlignment* = Aligement sur l'axe horizontal

*mainAxisSize* = largeur de la colonne

*crossAxisAlignment* = Aligement sur l'axe vertical





# Widgets Layout : *Stack*

Le widget *Stack* permet de disposer les widgets en son sein les uns sur les autres. Il possède un seul axe. Le widget *Stack* n'a donc pas de colonne ni de ligne. Nous pouvons donc afficher du texte sur une image en utilisant. Chaque enfant d'un widget *Stack* est soit positionné, soit non positionné.

Stack



## Propriétés :

**alignment** = Positionnement des widgets enfants



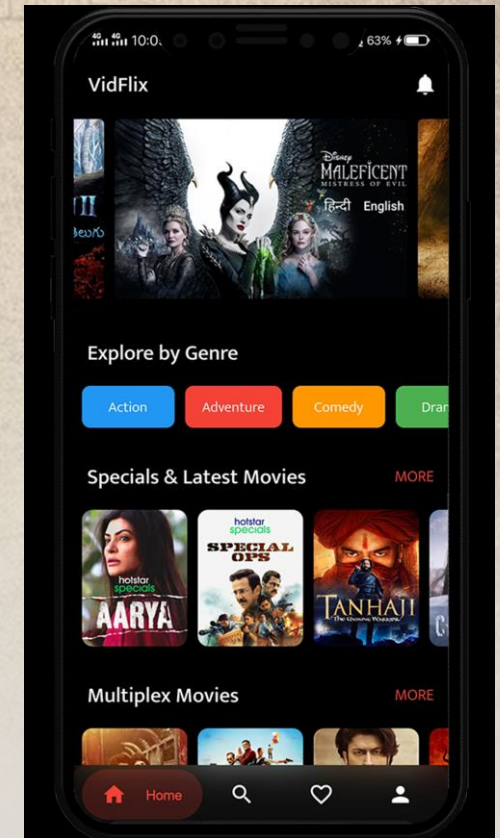
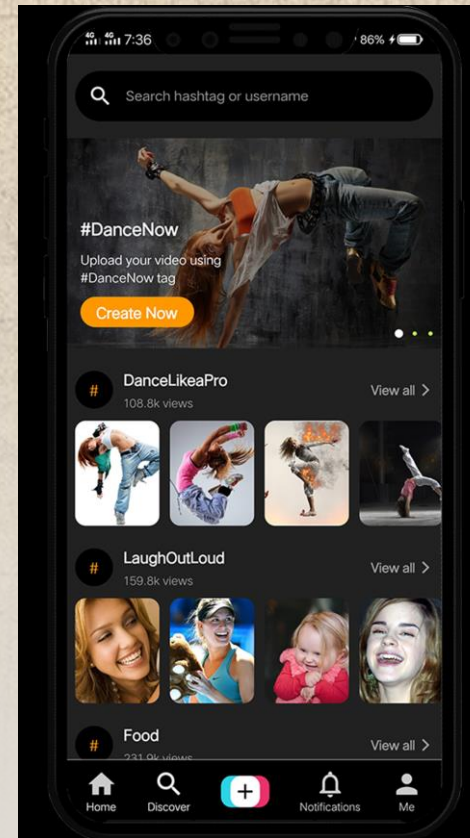
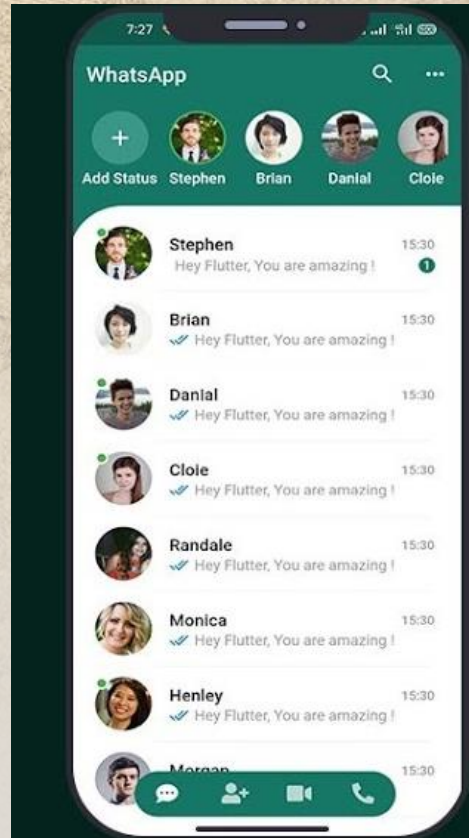
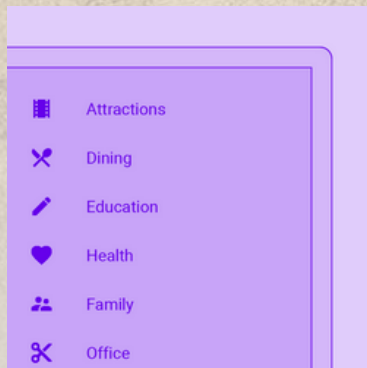


# Widgets Layout (View)

*ListView* est une liste déroulante de widgets disposés linéairement. La création d'un widget de liste déroulante dans Flutter est assez simple à l'aide du widget *ListView*. L'axe de la liste view est de terminé par la propriété *scrollDirection* (verticale ou horizontale). Les différents constructeurs de liste sont :

1. *ListView*
2. *ListView.Builder*
3. *ListView.Separated*
4. *ListView.Custom*

## Listview





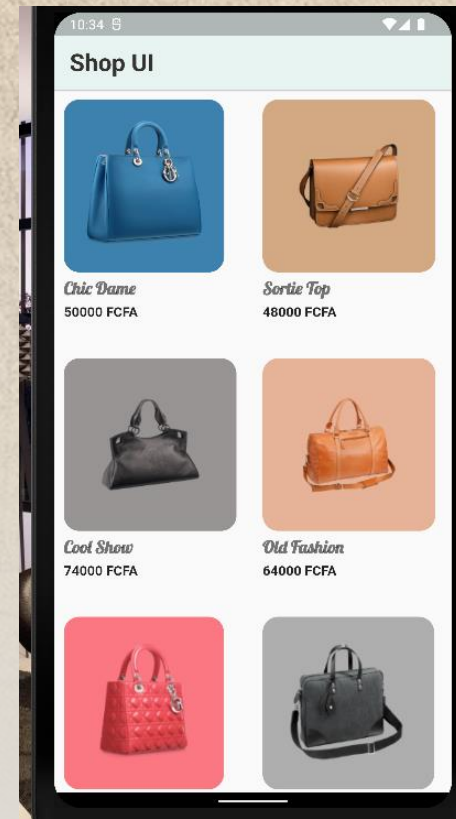


# Widgets Layout (View)

*GridView* permet de disposer les widgets en grille. Le widget *GridView* permet d'avoir des colonnes et des lignes, sous forme de grilles avec des éléments qui permettent de faire défiler. Dans *GridView*, nous pouvons également utiliser *List.generate* pour créer des widgets enfants en fonction du nombre de données. Nous avons les *GridView* suivants :

1. *GridView*
2. *GridView.count*
3. *GridView.builder*
4. *GridView.custom*
5. *GridView.extent*

## Gridview







# Widgets Container

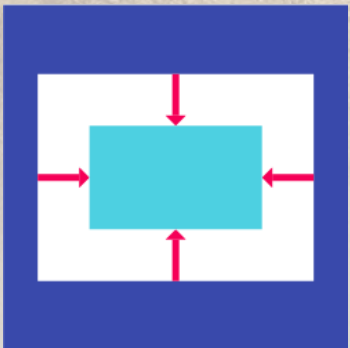
Les Container sont des widgets d'affichage et de disposition du contenu. Ils permettent de modifier l'apparence et la mise en page d'un widget. De ce fait ils peuvent embarquer un seul widget. Ils attendent donc un widget dans la propriété **child**.

**child** :Widget(...)

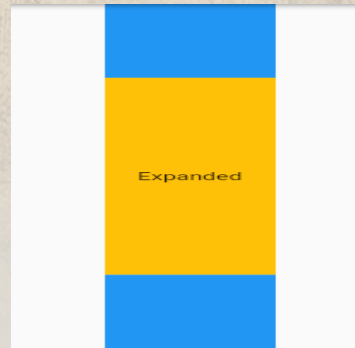
Ils peuvent gérer **le style, le positionnement, la bordure, les marges, etc.** du widget enfant. La propriété *décoration* est la plus utile des widgets *Container*.

*decoration* → BoxDecoration(...)

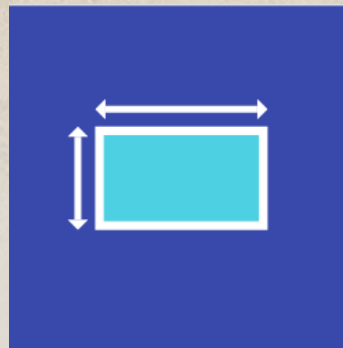
Center



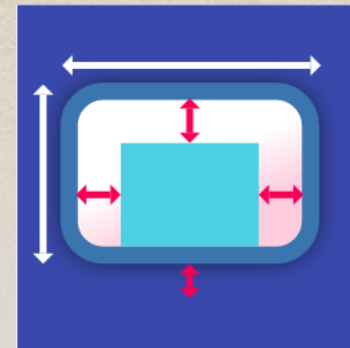
Expanded



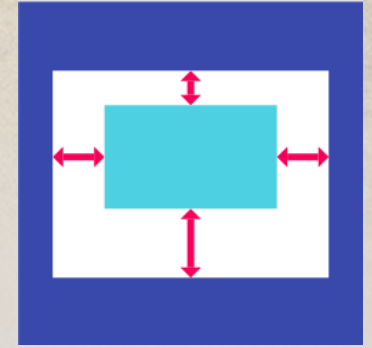
Sizebox



Container



Padding





# Widgets de contenu

## Text

Hello, Ruth! How are you?

text  
textAlign  
overflow

## Icon



Icon  
colors  
Size

## Image



Image.asset en local  
Image.network sur internet  
Image.file dans un fichier





# Widgets de contenu : *Text*

Le widget *Text* permet d'afficher du texte dans l'interface utilisateur d'une application mobile. En fait, le texte est une partie importante de toute application mobile, il est donc essentiel de s'assurer qu'il est bien placé et facile à lire. Pour ce faire *Text* contient des propriétés de style de texte (alignement, couleur, taille, police,...).

Text

Hello, Ruth! How are you?

```
Text(  
  String data,  
  {  
    Key key,  
    TextStyle style,  
    StrutStyle strutStyle,  
    TextAlign textAlign,  
    TextDirection textDirection,  
    Locale locale,  
    bool softWrap,  
    TextOverflow overflow,  
    double textScaleFactor,  
    int maxLines,  
    String semanticsLabel,  
    TextWidthBasis textWidthBasis,  
    TextHeightBehavior textHeightBehavior  
  }  
);
```





# Widgets de contenu : *Image*

Le widget *Image* permet d'afficher une image dans l'interface utilisateur d'une application mobile. *Image* peut afficher des image de 3 manières :

1. Via les assets [Image.asset](#) en local
2. Via internet [Image.network](#) sur internet
3. Via un fichier. [Image.file](#) dans un fichier

Image



Les assets sont des fichiers image qui sont ajoutés au dossier de l'application. Ensuite, ils seront utilisés pour afficher des images sans connexion Internet. Voici les étapes que vous devez suivre :

1. Créer un nouveau dossier appelé **/assets** dans votre projet.
2. Mettre les images dans le dossier.
3. **Définir les assets à utiliser dans pubspec.yaml.**
4. Charger l'image à partir des assets dans le code





# Widgets User Input

elevatedButton

Enregistrer

style: style,  
onPressed: () {},  
child: Widget()

outlinedButton

Enregistrer

style: style,  
onPressed: () {},  
child: Widget()

textField

Password

obscureText  
decoration  
onSubmitted  
Controller  
keyBoardType





# Widgets Gesture Controller

---

Les gestes sont principalement un moyen pour un utilisateur d'interagir avec une application mobile (ou tout autre appareil tactile). Les gestes sont généralement définis comme toute action / mouvement physique d'un utilisateur dans l'intention d'activer un contrôle spécifique de l'appareil mobile.

- **Tap** - Toucher la surface avec le bout du doigt pendant une courte période, puis relâcher le bout du doigt.
- **Double Tap** - Tapotez deux fois en peu de temps.
- **Drag** - Toucher la surface de l'appareil avec le bout du doigt puis bouger le bout du doigt de manière régulière et enfin relâcher le bout du doigt.
- **Flick** - Similaire au glisser, mais en le faisant de manière plus rapide.
- **Pinch** - Pincer la surface de l'appareil à l'aide de deux doigts.
- **Spread/Zoom** - Face au pincement.
- **Panning** - Toucher la surface du bout du doigt et le déplacer dans n'importe quelle direction sans relâcher le bout du doigt.

Flutter fournir deux widgets pour la prise en compte des gestes : ***GestureDetector & InkWell***



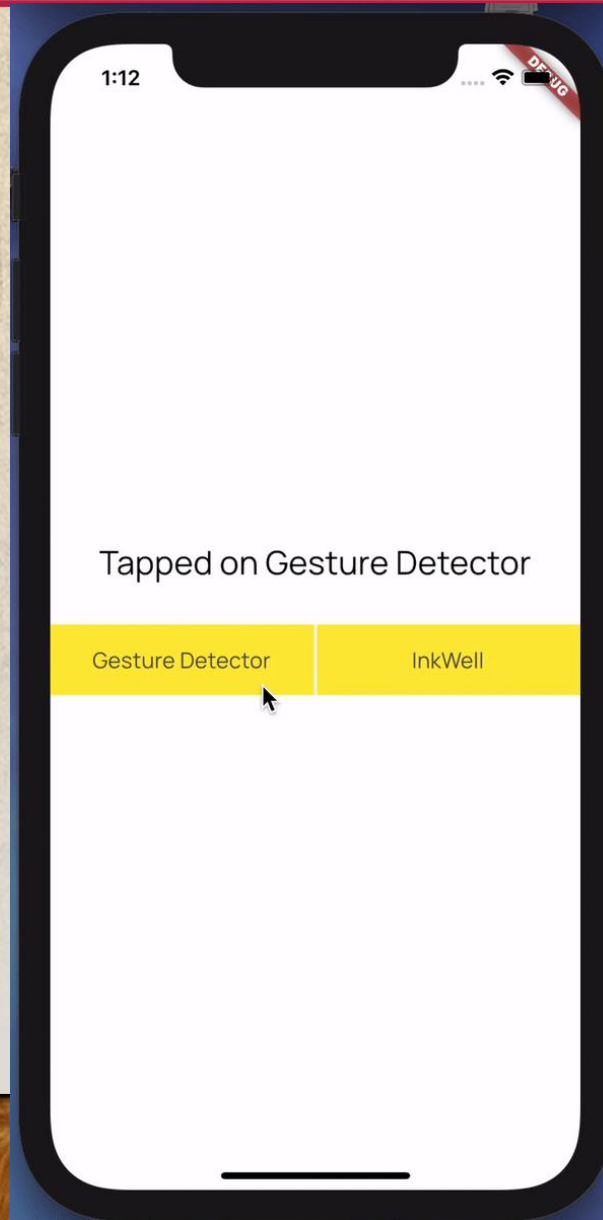


# Widgets Gesture Controller

## GestureDetector

Beaucoup plus des gestes pris en compte

- onDoubleTap
- onHorizontalDragDown
- onLongPress
- onTap
- onTapCancel
- onTapDown
- onTapUp
- onVerticalDragDown
- ...



## Inkwell

Retour visuel du geste

- onDoubleTap
- onLongPress
- onTap
- onFocusChange
- ...





# Styliser les Widgets

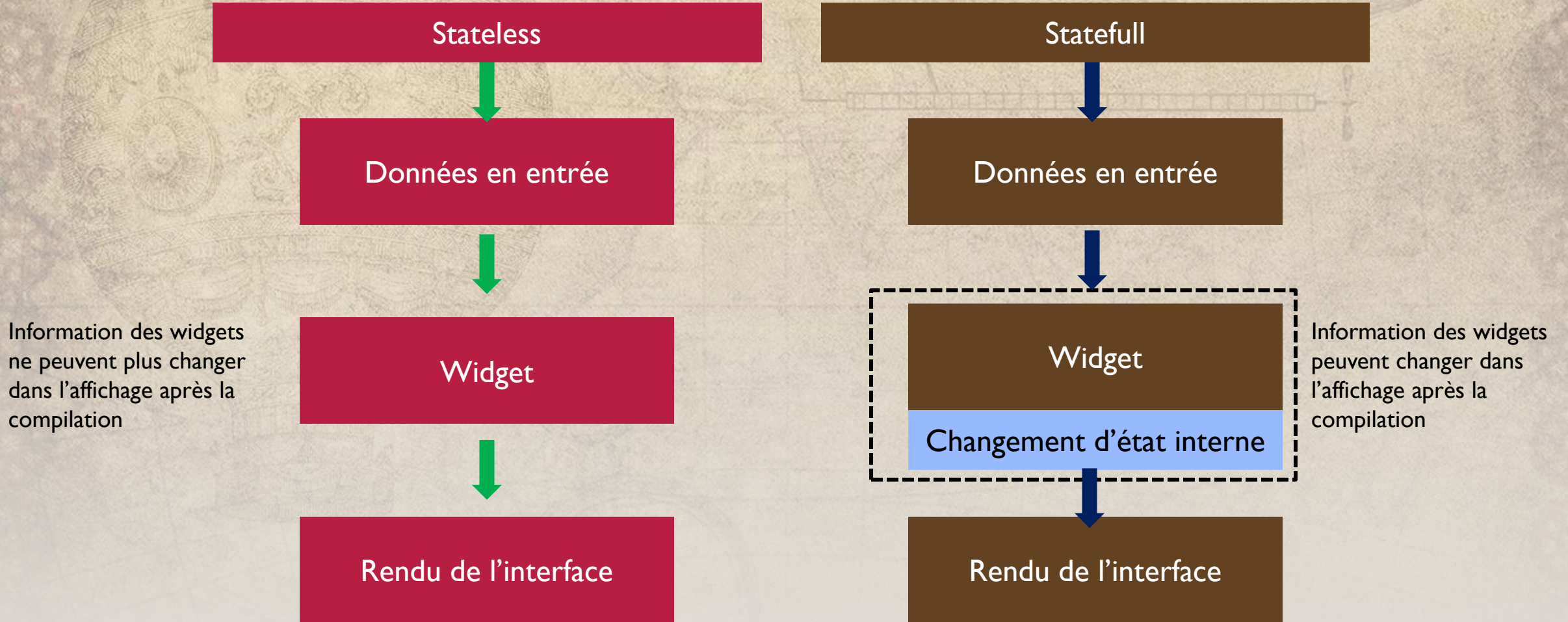
## Propriétés de style

- Decoration
- Theme
- mediaquery
- Google font package
- Border
- Border raduis





# Interactivité : stateless vs statefull







# Navigation

